## TALLER APLICADO

# CAPTURA OFFLINE-FIRST Y SINCRONIZACIÓN MANUAL CON FIREBASE RTDB

# 1. Situación problema

Una organización requiere recolectar información en campo mediante dispositivos móviles. Las actividades pueden desarrollarse en zonas con conectividad limitada, intermitente o inexistente, por lo que el registro no puede depender de una conexión activa.

Cada equipo deberá proponer un contexto propio de captura. El sistema deberá permitir diligenciar información, asociar como mínimo una imagen, guardar múltiples registros localmente, cerrar y volver a abrir la aplicación sin perder datos y, posteriormente, transferir únicamente los registros pendientes a Firebase Realtime Database cuando el usuario lo ordene.

La solución deberá combinar calidad técnica y calidad visual. No será suficiente demostrar que los datos llegan a Firebase: se evaluará el diseño del flujo, la coherencia del modelo de datos, la estrategia de persistencia, la prevención de duplicados, la respuesta ante errores y la experiencia del usuario.

# 2. Objetivo general

Diseñar y desarrollar una aplicación móvil con enfoque Offline-First que capture, valide y almacene información localmente, gestione al menos una evidencia fotográfica y permita sincronizar manualmente los registros pendientes con Firebase Realtime Database mediante una acción explícita del usuario.

# 3. Condiciones obligatorias

- Grupos de 2 a 4 estudiantes.
- Cada registro deberá contener mínimo 15 datos o atributos.
- De los 15 datos, mínimo 8 deberán ser diligenciados o seleccionados directamente por el usuario.
- Cada registro deberá incluir mínimo 1 imagen capturada con cámara o seleccionada desde el dispositivo.
- La aplicación deberá permitir trabajar sin Internet desde el inicio y durante toda la captura.
- El botón GUARDAR deberá almacenar localmente; no deberá escribir en Firebase.
- La comunicación con Firebase RTDB se realizará solamente cuando el usuario pulse SINCRONIZAR.
- Los registros deberán conservarse aunque la aplicación se cierre y se vuelva a abrir.
- El sistema deberá distinguir, como mínimo, entre registros PENDIENTES y SINCRONIZADOS.
- Una segunda sincronización no deberá duplicar registros ya enviados.
- Cada integrante deberá demostrar aportes propios mediante evidencias verificables.

# 4. Fase 1 - Definición del caso de captura

Antes de iniciar la programación, el grupo deberá construir una ficha de identidad del proyecto. Esta ficha será la referencia para comprobar que las pantallas, los datos y las validaciones implementadas corresponden realmente al problema elegido.

| | |
|---|---|
| **Nombre de la aplicación** | |
| **Usuario principal** | ¿Quién utilizará la aplicación? |
| **Lugar o contexto de uso** | ¿Dónde y bajo qué condiciones se capturan los datos? |
| **Problema concreto** | ¿Qué situación necesita ser resuelta? |
| **Objetivo de captura** | ¿Qué decisión o proceso se beneficiará de estos datos? |
| **Razón para trabajar offline** | Describa una situación realista en la que no exista conectividad. |

| | |
|---|---|
| **Resultado esperado** | ¿Qué deberá poder analizarse o consultar después de la captura? |
| **Criterio de éxito** | ¿Cómo sabrá el grupo que su solución resuelve el problema? |

**Producto de esta fase:** una descripción de máximo una página y un objetivo redactado por el grupo con la forma: "Nuestra aplicación permitirá capturar _____ para _____ en el contexto de _____."

# 5. Fase 2 - Matriz de decisiones del grupo

Antes de programar, el equipo deberá acordar y documentar las siguientes decisiones. Las decisiones deberán mantenerse coherentes durante la implementación y la sustentación.

| Decisión | Elección del grupo | Alternativa descartada | Justificación técnica o funcional |
|---|---|---|---|
| Identificador único del registro | | | |
| Forma de almacenar la imagen localmente | | | |
| Reglas para declarar un registro válido | | | |
| Qué campos serán obligatorios | | | |
| Qué ocurre si falla una sincronización a mitad del proceso | | | |
| Cómo se determinará que un registro ya fue enviado | | | |
| Cómo se informará al usuario que está trabajando offline | | | |
| Cómo se mostrará el estado PENDIENTE/SINCRONIZADO | | | |
| Qué información podrá editarse antes de sincronizar | | | |
| Criterio para limpiar o conservar datos ya sincronizados | | | |

# 6. Fase 3 - Diseño de los 15 datos

Cada equipo deberá construir su propio diccionario de datos. No se aceptará una lista de campos sin explicar para qué se utilizan, qué valores reciben y cómo se validan.

| # | Dato | Tipo | Origen | Obligatorio | Regla de validación | Uso dentro del problema |
|---|---|---|---|---|---|---|
| 1 | Ej.: idRegistro | Texto/UUID | Automático | Sí | Único | Evitar duplicados |
| 2 | Ej.: evidenciaFotográfica | Imagen | Usuario | Sí | Debe existir antes de guardar | Soporta la evidencia del caso |
| 3 | | | | | | |
| 4 | | | | | | |
| 5 | | | | | | |
| 6 | | | | | | |
| 7 | | | | | | |
| 8 | | | | | | |
| 9 | | | | | | |
| 10 | | | | | | |
| 11 | | | | | | |
| 12 | | | | | | |
| 13 | | | | | | |
| 14 | | | | | | |
| 15 | | | | | | |

**Regla:** al menos 8 filas deberán tener como origen “Usuario” y al menos 1 fila deberá corresponder a una imagen. Los restantes datos pueden generarse automáticamente, por ejemplo fecha, hora, identificador, estado de sincronización o fecha de envío.

# 7. Reglas de validación propias

El grupo deberá definir mínimo 5 reglas de validación específicas de su caso. No cuentan como reglas distintas expresiones genéricas como “no dejar vacío” repetidas en varios campos.

| Regla | Condición | Mensaje al usuario | Por qué es necesaria |
|---|---|---|---|
| R1 | | | |
| R2 | | | |
| R3 | | | |
| R4 | | | |
| R5 | | | |

# 8. Fase 4 - Arquitectura funcional obligatoria

La arquitectura deberá separar claramente la captura local de la transferencia remota. El grupo deberá poder explicar dónde está cada dato en cada momento del proceso.

1. Usuario diligencia el formulario y selecciona/captura imagen
2. La aplicación valida las reglas definidas por el grupo
3. **GUARDAR almacena el registro y la referencia de la imagen localmente**
4. El registro queda marcado como PENDIENTE
5. El usuario puede continuar creando registros sin Internet
6. **SINCRONIZAR consulta únicamente los registros pendientes y verifica conectividad**
7. **Los envíos confirmados cambian a SINCRONIZADO; los fallidos permanecen PENDIENTES**

## 9. Persistencia local

Se recomienda Room Database o una solución local equivalente que asegure persistencia real. Guardar información solamente en variables de memoria no cumple el taller. La fotografía deberá quedar disponible después de cerrar y abrir nuevamente la aplicación; por ello el equipo deberá decidir cómo conservará una copia o referencia persistente.

## 10. Estados mínimos de un registro

| Estado | Significado | Acción esperada |
|---|---|---|
| PENDIENTE | Guardado localmente y aún no confirmado en Firebase | Disponible para sincronización |
| SINCRONIZADO | Firebase confirmó su almacenamiento | No volver a crear un duplicado |
| ERROR (opcional) | Ocurrió un fallo controlado durante el envío | Mostrar causa y permitir reintento |

## 11. Política de sincronización

El grupo deberá describir el algoritmo de sincronización en lenguaje natural o pseudocódigo antes de implementarlo. El algoritmo deberá contemplar como mínimo: selección de pendientes, verificación de conexión, envío individual o por lotes, confirmación, actualización del estado local y continuidad ante fallos parciales.

| | |
|---|---|
| **Entrada al proceso** | |
| **Condición de inicio** | |
| **Selección de registros** | |
| **Acción de envío** | |

| | |
|---|---|
| **Confirmación** | |
| **Tratamiento del error** | |
| **Condición de finalización** | |

### 12. Firebase Realtime Database

Firebase RTDB será el repositorio remoto de los registros sincronizados. Cada equipo deberá proponer una estructura jerárquica coherente con su caso y explicar cómo el identificador local se relaciona con la llave utilizada en Firebase. Para la imagen, el equipo deberá definir y justificar una estrategia compatible con el alcance del taller, evitando enviar archivos innecesariamente grandes y manteniendo la copia local mientras el registro esté pendiente.

## 13. Fase 5 - Diseño visual y experiencia de usuario

La aplicación deberá presentar un nivel visual alto. Antes de programar, cada grupo deberá crear un boceto propio de las pantallas y definir un pequeño sistema visual.

| Elemento | Definición del grupo | Justificación |
|---|---|---|
| Paleta principal | | |
| Color para PENDIENTE | | |
| Color para SINCRONIZADO | | |
| Tipografía/tamaños | | |
| Estilo de tarjetas o contenedores | | |
| Iconografía | | |
| Patrón de mensajes y confirmaciones | | |

### 14. Pantallas mínimas

- Inicio/Resumen: deberá mostrar, como mínimo, cantidad de registros locales, pendientes y sincronizados, además de accesos a nuevo registro, listado y sincronización.
- Captura: deberá agrupar los campos por significado, no como una lista plana. Debe incluir validaciones visibles, evidencia fotográfica y una acción clara para guardar localmente.
- Listado local: deberá permitir reconocer cada registro, su fecha o identificador y su estado de sincronización.
- Detalle o revisión: deberá permitir verificar la información registrada, incluyendo la imagen, antes o después de sincronizar según la lógica definida por el grupo.

### 15. Boceto obligatorio antes del código

El equipo deberá anexar un boceto inicial realizado antes de la implementación. Puede ser elaborado a mano o mediante una herramienta de prototipado. Debe incluir anotaciones que indiquen qué componentes se usarán y por qué están ubicados de esa manera. Durante la entrega se comparará este boceto con la versión final y el grupo explicará los cambios realizados.

# 16. Fase 6 - Casos de comportamiento

El grupo deberá completar esta matriz antes de finalizar la aplicación. Cada respuesta deberá corresponder a lo implementado, no a una respuesta teórica.

| Situación | Comportamiento decidido por el grupo | Evidencia de prueba |
|---|---|---|
| No hay Internet al abrir la app | | |
| Se guardan 4 registros sin conexión | | |
| Se cierra la aplicación y se vuelve a abrir | | |
| El usuario pulsa SINCRONIZAR sin Internet | | |
| Hay 7 pendientes y falla el envío del cuarto | | |
| Se pulsa SINCRONIZAR dos veces | | |
| La imagen seleccionada deja de estar accesible | | |
| Un registro no cumple una regla de validación | | |
| Firebase confirma solo parte de los registros | | |
| El usuario intenta editar un registro ya sincronizado | | |

# 17. Pruebas obligatorias

**Prueba A - Persistencia completamente offline:** Desactivar Wi-Fi y datos móviles; abrir la app; crear dos registros con imagen; cerrar completamente la aplicación; abrir de nuevo; verificar que ambos registros y sus imágenes continúan disponibles.

**Prueba B - Intento de sincronización sin conectividad:** Con registros pendientes y sin Internet, pulsar SINCRONIZAR. La aplicación deberá informar el problema sin cerrarse ni modificar incorrectamente el estado de los registros.

**Prueba C - Sincronización manual:** Activar Internet y pulsar SINCRONIZAR. Verificar en Firebase RTDB que los registros correctos fueron almacenados y que localmente cambian a SINCRONIZADO.

**Prueba D - Prevención de duplicados:** Pulsar nuevamente SINCRONIZAR. Los registros ya confirmados no deberán aparecer duplicados en RTDB.

**Prueba E - Fallo parcial:** Provocar o simular un error durante el envío y demostrar que los registros no confirmados continúan pendientes mientras los confirmados conservan su estado correcto.

# 18. Evidencia de trabajo individual

La calificación del producto es grupal, pero la participación se verificará individualmente. Cada estudiante deberá dejar trazabilidad suficiente para demostrar qué construyó, qué problema resolvió y qué decisiones puede explicar.

| Integrante | Responsabilidad principal | 3 evidencias mínimas | Problema técnico resuelto | Parte que explicará en sustentación |
|---|---|---|---|---|
| Integrante 1 | | | | |
| Integrante 2 | | | | |
| Integrante 3 | | | | |
| Integrante 4 | | | | |

**Evidencias válidas:** commits, capturas del proceso, historial de cambios, pruebas realizadas, diagramas, registro de errores, comparación antes/después, explicación presencial de una parte del código o video breve mostrando el funcionamiento del componente propio.

# 19. Prueba de adaptación en vivo

Durante la sustentación, el docente seleccionará una decisión ya documentada por el equipo y solicitará un ajuste pequeño sobre la aplicación o su lógica. El cambio podrá relacionarse, por ejemplo, con una validación, un estado, un campo, un mensaje, el orden de un elemento visual o una condición de sincronización. El objetivo es comprobar que el equipo comprende su propia solución y puede mantenerla.

| Elemento susceptible de ajuste | Dónde está definido | Quién del grupo puede modificarlo |
|---|---|---|
| Regla de validación | | |
| Campo del formulario | | |
| Estado o mensaje | | |
| Comportamiento de sincronización | | |

# 20. Entregables

1. Proyecto completo de Android Studio.
2. APK funcional.
3. Ficha de identidad del proyecto y objetivo de captura.
4. Matriz de decisiones del grupo completamente diligenciada.
5. Diccionario de los 15 datos y mínimo 5 reglas de validación propias.
6. Boceto inicial de pantallas con anotaciones y evidencia del diseño final.
7. Diagrama de flujo de captura, almacenamiento local y sincronización.
8. Evidencias de las cinco pruebas obligatorias.
9. Captura de la estructura de Firebase RTDB con datos sincronizados.
10. Tabla de participación individual y evidencias de cada integrante.
11. Bitácora breve de problemas encontrados y soluciones aplicadas.
12. Sustentación funcional en el dispositivo o emulador.

# 21. Rúbrica de evaluación

| Criterio | Valor | Evidencia principal |
|---|---|---|
| Planteamiento del problema y coherencia del objetivo | 8% | |

| | | |
|---|---|---|
| Modelo de datos, reglas y decisiones del grupo | 12% | |
| Funcionamiento Offline-First y persistencia local | 18% | |
| Captura y persistencia de imagen | 8% | |
| Sincronización manual con Firebase RTDB | 15% | |
| Prevención de duplicados y manejo de fallos | 10% | |
| Diseño visual, navegación y experiencia de usuario | 12% | |
| Pruebas y evidencias técnicas | 7% | |
| Participación individual y dominio en sustentación | 10% | |
| **TOTAL** | **100%** | |

## 22. Condiciones críticas

| | |
|---|---|
| **La aplicación registra directamente en Firebase al diligenciar o guardar** | No cumple la arquitectura central del taller. |
| **Los datos se pierden al cerrar la app** | No cumple persistencia offline. |
| **No puede abrirse o capturar sin Internet** | No cumple el enfoque Offline-First. |
| **Una segunda sincronización duplica información** | La estrategia de identificación/sincronización está incompleta. |
| **La interfaz no refleja estados ni errores** | La experiencia de usuario está incompleta. |
| **Las decisiones documentadas no coinciden con la aplicación** | El grupo deberá justificar la diferencia y demostrar dónde está implementada la lógica real. |
| **Un integrante no puede explicar su aporte** | Su valoración individual podrá diferir de la nota del producto grupal. |

## 23. Preguntas para la sustentación

1. ¿Dónde queda físicamente la información cuando no existe Internet?
2. ¿Qué dato identifica de forma única un registro y por qué eligieron esa estrategia?
3. ¿Qué ocurre exactamente desde que se pulsa GUARDAR hasta que aparece el estado PENDIENTE?
4. ¿Cómo conserva la aplicación la imagen después de cerrarse?
5. ¿Cómo seleccionan solamente los registros que faltan por enviar?
6. ¿Qué pasa si Firebase confirma unos registros y otros fallan?

7. ¿Cómo comprobaron que una segunda sincronización no crea duplicados?
8. Explique una regla de validación que sea específica del problema elegido.
9. ¿Qué cambio hicieron entre el boceto inicial y la interfaz final y por qué?
10. ¿Qué decisión técnica fue discutida por el grupo antes de implementarla?
11. ¿Qué error real apareció durante el desarrollo y cómo lo diagnosticaron?
12. ¿Qué parte desarrolló usted y dónde puede demostrarlo en el proyecto?

## 24. Criterio de cierre del taller

**CAPTURAR → VALIDAR → GUARDAR LOCALMENTE → PENDIENTE**

Solo cuando el usuario lo decida y exista conectividad:

**SINCRONIZAR → CONFIRMAR EN FIREBASE RTDB → ACTUALIZAR ESTADO LOCAL**

*El propósito del taller es construir una solución que pueda explicarse, probarse, mantenerse y adaptarse. La calidad final dependerá tanto del funcionamiento técnico como de la coherencia entre el problema elegido, las decisiones tomadas por el grupo y la implementación presentada.*